

Network Working Group
Internet-Draft
Intended status: Informational
Expires: 18 October 2026

C. Celi, Ed.
16 April 2026

ACVP ML-KEM JSON Specification
draft-celi-acvp-ml-kem-01

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at <https://datatracker.ietf.org/drafts/current/>.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 18 October 2026.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (<https://trustee.ietf.org/license-info>) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document.

Table of Contents

1. Acknowledgements	2
2. Abstract	2
3. Introduction	3
4. Conventions	3
4.1. Notation conventions	3
4.2. Terms and Definitions	3
4.2.1. Prompt	3
4.2.2. Registration	3
4.2.3. Response	4
4.2.4. Test Case	4
4.2.5. Test Group	4
4.2.6. Test Vector Set	4
4.2.7. Validation	4
5. Supported ML-KEM Algorithms	4
6. Test Types and Test Coverage	4
6.1. Test Types	4
6.2. Test Coverage	6
6.2.1. Requirements Covered	6

6.2.2. Requirements Not Covered	6
7. Capabilities Registration	6
7.1. Prerequisites	7
7.2. Required Prerequisite Algorithms for ML-KEM Validations	8
7.3. ML-KEM Algorithm Registration Properties	8
7.3.1. ML-KEM keyGen Mode Capabilities Example	9
7.3.2. ML-KEM encapDecap Mode Capabilities Example	10
8. Test Vectors	10
8.1. ML-KEM keyGen Test Vectors	11
8.1.1. ML-KEM keyGen Test Groups JSON Schema	12
8.1.2. ML-KEM keyGen Test Case JSON Schema	12
8.2. ML-KEM encapDecap Test Vectors	13
8.2.1. ML-KEM encapDecap Test Groups JSON Schema	14
8.2.2. ML-KEM encapDecap Test Case JSON Schema	14
9. Test Vector Responses	16
9.1. ML-KEM keyGen Test Vector Responses	18
9.2. ML-KEM encapDecap Test Vector Responses	18
10. Security Considerations	21
11. IANA Considerations	21
12. Normative References	21
Author's Address	21

1. Acknowledgements

There are no acknowledgements.

2. Abstract

This document defines the JSON schema for testing Module Lattice-based Key Encapsulation Mechanism (ML-KEM) implementations with the ACVP specification.

Celi

Expires 18 October 2026

[Page 2]

Internet-Draft

ACVP ML-KEM

April 2026

3. Introduction

The Automated Crypto Validation Protocol (ACVP) defines a mechanism to automatically verify the cryptographic implementation of a software or hardware crypto module. The ACVP specification defines how a crypto module communicates with an ACVP server, including crypto capabilities negotiation, session management, authentication, vector processing and more. The ACVP specification does not define algorithm specific JSON constructs for performing the crypto validation. A series of ACVP sub-specifications define the constructs for testing individual crypto algorithms. Each sub-specification addresses a specific class of crypto algorithms. This sub-specification defines the JSON constructs for testing Module Lattice-based Key Encapsulation Mechanism (ML-KEM) implementations using ACVP.

This document is a work-product of the NIST (<https://www.nist.gov/>) Cryptographic Algorithm Validation Program (<https://csrc.nist.gov/projects/cryptographic-algorithm-validation-program>) (CAVP). Please communicate questions and feedback concerning this document to the CAVP by opening an issue on the [usnistgov/ACVP](https://github.com/usnistgov/ACVP) GitHub repository at

<https://github.com/usnistgov/ACVP/issues>.

4. Conventions

4.1. Notation conventions

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "*NOT RECOMMENDED*", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP 14 of [RFC2119] and [RFC8174] when, and only when, they appear in all capitals, as shown here.

4.2. Terms and Definitions

4.2.1. Prompt

JSON sent from the server to the client describing the tests the client performs

4.2.2. Registration

The initial request from the client to the server describing the capabilities of one or several algorithm, mode and revision combinations

Celi	Expires 18 October 2026	[Page 3]
Internet-Draft	ACVP ML-KEM	April 2026

4.2.3. Response

JSON sent from the client to the server in response to the prompt

4.2.4. Test Case

An individual unit of work within a prompt or response

4.2.5. Test Group

A collection of test cases that share similar properties within a prompt or response

4.2.6. Test Vector Set

A collection of test groups under a specific algorithm, mode, and revision

4.2.7. Validation

JSON sent from the server to the client that specifies the correctness of the response

5. Supported ML-KEM Algorithms

The following ML-KEM algorithms MAY be advertised by the ACVP compliant cryptographic module. The list is in the form "algorithm / mode / revision".

* ML-KEM / keyGen / FIPS203

* ML-KEM / encapDecap / FIPS203

6. Test Types and Test Coverage

6.1. Test Types

The ACVP server performs a set of tests on the specified ML-KEM algorithm in order to assess the correctness and robustness of the implementation. A typical ACVP validation session SHALL require multiple tests to be performed for every supported permutation of ML-KEM capabilities. This section describes the design of the tests used to validate implementations of the ML-KEM algorithms.

Celi Expires 18 October 2026 [Page 4]
Internet-Draft ACVP ML-KEM April 2026

- * ML-KEM / keyGen / * "AFT" - Algorithm Functional Test. The IUT is REQUIRED for each test case provided, to generate a key pair from a provided seed. The key pair is communicated to the ACVP server and validated. This tests the implementation of Algorithm 16 ML-KEM.KeyGen_internal() from [FIPS203] Section 6.1.
- * ML-KEM / encapDecap / * "AFT" - Algorithm Functional Test. The server will generate a random m and encapsulation key ek. The IUT SHALL generate the corresponding shared secret k and ciphertext c. This tests the implementation of Algorithm 17 ML-KEM.Encaps_internal() from [FIPS203] Section 6.2.
- * ML-KEM / encapDecap / * "VAL" - Validation Test. There are several variations for this test type depending on the function being tested.
 - For decapsulation, the server will generate a decapsulation key dk and a potentially valid or invalid ciphertext c. The IUT SHALL generate the corresponding shared secret k whether the k is provided from the valid decapsulation route or from the implicit rejection present in [FIPS203] Algorithm 17. This tests the implementation of Algorithm 18 ML-KEM.Decaps_internal() from [FIPS203] Section 6.3.
 - For the encapsulation key check, the server will generate a potentially valid or invalid encapsulation key ek. The IUT SHALL determine if the provided ek is valid based on [FIPS203] Section 7.2.
 - For the decapsulation key check, the server will generate a potentially valid or invalid decapsulation key dk. The IUT SHALL determine if the provided dk is valid based on [FIPS203] Section 7.3.

The decapsulation modifications are:

- * "valid decapsulation" - No modification to the ciphertext is made and the decapsulation will follow the valid path to generate a shared key k.
- * "modified ciphertext" - The ciphertext is randomly modified and the decapsulation will follow the implicit rejection path to generate a shared key k.

The encapsulation key check modifications are:

Celi Expires 18 October 2026 [Page 5]
Internet-Draft ACVP ML-KEM April 2026

- * "valid encapsulation key" – No modification to the encapsulation key is made. The encapsulation key will pass the encapsulation key check.
- * "noisy linear system values too large" – The encapsulation key is modified such that a value in the noisy linear system is encoded into the key as a value greater than Q . This encapsulation key will not pass the encapsulation key check.

The decapsulation key check modifications are:

- * "valid decapsulation key" – No modification to the decapsulation key is made. The decapsulation key will pass the decapsulation key check.
- * "modified H " – The decapsulation key is modified such that the concatenated values $ek || H(ek)$ will no longer match by modifying $H(ek)$ to be a different value. This decapsulation key will not pass the decapsulation key check.

6.2. Test Coverage

The tests described in this document have the intention of ensuring an implementation is conformant to [FIPS203].

6.2.1. Requirements Covered

- * The tests will ensure conformity and correctness of an implementation of the algorithms supported.

6.2.2. Requirements Not Covered

- * FIPS 203 Section 3.3. Requirements for ML-KEM Implementations. Requirements outlined in this section are not testable by an ACVP server. An ACVP server will not test the zeroization of intermediate values, security strength of the deterministic random bit generators (DRBGs), or incorrect length signatures or public keys.

7. Capabilities Registration

ACVP requires crypto modules to register their capabilities. This allows the crypto module to advertise support for specific algorithms, notifying the ACVP server which algorithms need test vectors generated for the validation process. This section describes the constructs for advertising support of ML-KEM algorithms to the ACVP server.

Celi Expires 18 October 2026 [Page 6]
Internet-Draft ACVP ML-KEM April 2026

The algorithm capabilities MUST be advertised as JSON objects within the 'algorithms' value of the ACVP registration message. The 'algorithms' value is an array, where each array element is an individual JSON object defined in this section. The 'algorithms' value is part of the 'capability_exchange' element of the ACVP JSON

registration message. See the ACVP specification [ACVP] for more details on the registration message.

7.1. Prerequisites

Each algorithm implementation MAY rely on other cryptographic primitives. For example, RSA Signature algorithms depend on an underlying hash function. Each of these underlying algorithm primitives must be validated, either separately or as part of the same submission. ACVP provides a mechanism for specifying the required prerequisites:

Prerequisites, if applicable, MUST be submitted in the registration as the prereqVals JSON property array inside each element of the algorithms array. Each element in the prereqVals array MUST contain the following properties

JSON Property	Description	JSON Type
algorithm	a prerequisite algorithm	string
valValue	algorithm validation number	string

Table 1: Prerequisite Properties

A "valValue" of "same" SHALL be used to indicate that the prerequisite is being met by a different algorithm in the capability exchange in the same registration.

An example description of prerequisites within a single algorithm capability exchange looks like this

```
"prereqVals":
[
  {
    "algorithm": "Alg1",
    "valValue": "Val-1234"
  },
  {
    "algorithm": "Alg2",
    "valValue": "same"
  }
]
```

7.2. Required Prerequisite Algorithms for ML-KEM Validations

Each ML-KEM implementation relies on other cryptographic primitives. For example, ML-KEM keyGen uses an underlying SHA algorithm. Each of these underlying algorithm primitives must be validated, either separately or as part of the same submission. ACVP provides a

mechanism for specifying the required prerequisites:

JSON Value	Description	JSON Type	Valid Values
algorithm	a prerequisite algorithm	string	SHA, or DRBG
valValue	algorithm validation number	string	Actual number or "same"
prereqAlgVal	prerequisite algorithm validation	object with algorithm and valValue properties	See above

Table 2: Required ML-KEM Prerequisite Algorithms JSON Values

7.3. ML-KEM Algorithm Registration Properties

Each ML-KEM algorithm capability advertised is a self-contained JSON object using the following values.

Celi

Expires 18 October 2026

[Page 8]

Internet-Draft

ACVP ML-KEM

April 2026

JSON Value	Description	JSON Type	Valid Values
algorithm	The ML-KEM algorithm to be validated	string	See Section 5
mode	The ML-KEM mode to be validated	string	See Section 5
revision	The algorithm testing revision to use	string	See Section 5
prereqVals	Prerequisite algorithm validations	array of prereqAlgVal objects	See Section 7.2
parameterSets	The ML-KEM parameter sets supported	array of strings	"ML-KEM-512", "ML-KEM-768", "ML-KEM-1024"
functions	The ML-KEM encapsulation or decapsulation modes supported	array of strings	"encapsulation", "decapsulation", "encapsulationKeyCheck", "decapsulationKeyCheck"

+-----+-----+-----+-----+-----+-----+

Table 3: ML-KEM Algorithm Capabilities JSON Values

Note: The functions property is only applicable to ML-KEM / encapDecap / FIPS203.

7.3.1. ML-KEM keyGen Mode Capabilities Example

Below is an example of the registration for ML-KEM / keyGen / FIPS203

Celi	Expires 18 October 2026	[Page 9]
Internet-Draft	ACVP ML-KEM	April 2026

```
{
  "algorithm": "ML-KEM",
  "mode": "keyGen",
  "revision": "FIPS203",
  "prereqVals": [
    {
      "algorithm": "SHA",
      "valValue": "123456"
    }
  ],
  "parameterSets": ["ML-KEM-512", "ML-KEM-768", "ML-KEM-1024"]
}
```

7.3.2. ML-KEM encapDecap Mode Capabilities Example

Below is an example of the registration for ML-KEM / encapDecap / FIPS203

```
{
  "algorithm": "ML-KEM",
  "mode": "encapDecap",
  "revision": "FIPS203",
  "prereqVals": [
    {
      "algorithm": "SHA",
      "valValue": "123456"
    }
  ],
  "parameterSets": ["ML-KEM-512", "ML-KEM-768", "ML-KEM-1024"]
  "functions": ["encapsulation", "decapsulation", "encapsulationKeyCheck",
"decapsulationKeyCheck"]
}
```

8. Test Vectors

The ACVP server provides test vectors to the ACVP client, which are then processed and returned to the ACVP server for validation. A typical ACVP validation test session would require multiple test vector sets to be downloaded and processed by the ACVP client. Each test vector set represents an individual cryptographic algorithm defined during the capability exchange. This section describes the JSON schema for a test vector set used with Module Lattice-based Key Encapsulation Mechanism (ML-KEM) algorithms.

The test vector set JSON schema is a multi-level hierarchy that contains meta data for the entire vector set as well as individual test vectors to be processed by the ACVP client. The following table describes the JSON elements at the top level of the hierarchy.

Celi Expires 18 October 2026 [Page 10]

Internet-Draft ACVP ML-KEM April 2026

JSON Values	Description	JSON Type
acvVersion	Protocol version identifier	string
vsId	Unique numeric vector set identifier	integer
algorithm	Algorithm defined in the capability exchange	string
mode	Mode defined in the capability exchange	string
revision	Protocol test revision selected	string
testGroups	Array of test group JSON objects. Depending on the algorithm, see Section 8.1.1 or Section 8.2.1	array

Table 4: Top Level Test Vector JSON Elements

An example of this would look like this

```
[
  {
    "acvVersion": <version>
  },
  {
    "vsId": 1,
    "algorithm": "Alg1",
    "mode": "Mode1",
    "revision": "Revision1.0",
    "testGroups": [ ... ]
  }
]
```

8.1. ML-KEM keyGen Test Vectors

Celi Expires 18 October 2026 [Page 11]

Internet-Draft ACVP ML-KEM April 2026

8.1.1. ML-KEM keyGen Test Groups JSON Schema

The testGroups element at the top level in the test vector JSON object is an array of test groups. Test vectors are grouped into similar test cases to reduce the amount of data transmitted in the vector set. For instance, all test vectors that use the same key size would be grouped together. The Test Group JSON object contains meta data that applies to all test vectors within the group. The following table describes the ML-KEM JSON elements of the Test Group JSON object.

The test group for ML-KEM / keyGen / FIPS203 is as follows:

JSON Value	Description	JSON type
tgId	Numeric identifier for the test group, unique across the entire vector set	integer
testType	The test operation performed	string
parameterSet	The ML-KEM parameter set used	string
tests	Array of individual test vector JSON objects, which are defined in Section 8.1.2	array

Table 5: ML-KEM keyGen Test Group JSON Object

8.1.2. ML-KEM keyGen Test Case JSON Schema

Each test group contains an array of one or more test cases. Each test case is a JSON object that represents a single test vector to be processed by the ACVP client. The following table describes the JSON elements for each ML-KEM test vector.

Celi Expires 18 October 2026 [Page 12]
 Internet-Draft ACVP ML-KEM April 2026

JSON Value	Description	JSON type
tcId	Numeric identifier for the test case, unique across the entire vector set	integer
d	The seed d used to generate the key pair	hex
z	The seed z used to generate the key pair	hex

Table 6: ML-KEM keyGen Test Case JSON Object

The following is an example JSON object sent from the server to the client for ML-KEM / keyGen / FIPS203.

```
[
  {
    "acvVersion": <acvp-version>
  },
  {
    "vsId": 1564,
    "algorithm": "ML-KEM",
    "mode": "keyGen",
    "revision": "FIPS203",
    "testGroups": [
      {
        "tgId": 1,
        "testType": "AFT",
        "parameterSet": "ML-KEM-512",
        "tests": [
          {
            "tcId": 1,
            "d": "C105DC2...",
            "z": "AD310AB..."
          }
        ]
      }
    ]
  }
]
```

8.2. ML-KEM encapDecap Test Vectors

8.2.1. ML-KEM encapDecap Test Groups JSON Schema

The testGroups element at the top level in the test vector JSON object is an array of test groups. Test vectors are grouped into similar test cases to reduce the amount of data transmitted in the vector set. For instance, all test vectors that use the same key size would be grouped together. The Test Group JSON object contains meta data that applies to all test vectors within the group. The following table describes the ML-KEM JSON elements of the Test Group JSON object.

The test group for ML-KEM / encapDecap / FIPS203 is as follows:

JSON Value	Description	JSON type
tgId	Numeric identifier for the test group, unique across the entire vector set	integer
testType	The test operation performed	string
parameterSet	The ML-KEM parameter set used	string
function	Whether the tests are running encapsulation or decapsulation, or a	string

	key check	
tests	Array of individual test vector JSON objects, which are defined in Section 8.2.2	array

Table 7: ML-KEM encapDecap Test Group JSON Object

8.2.2. ML-KEM encapDecap Test Case JSON Schema

Each test group contains an array of one or more test cases. Each test case is a JSON object that represents a single test vector to be processed by the ACVP client. The following table describes the JSON elements for each ML-KEM test vector.

Celi Expires 18 October 2026 [Page 14]
Internet-Draft ACVP ML-KEM April 2026

JSON Value	Description	JSON type
tcId	Numeric identifier for the test case, unique across the entire vector set	integer
dk	The decapsulation key when the function is "decapsulation" or "decapsulationKeyCheck"	hex
ek	The encapsulation key when the function is "encapsulation" or "encapsulationKeyCheck"	hex
m	The random value used to derive the shared key when the function is "encapsulation"	hex
c	The cipher text when the function is "decapsulation"	hex

Table 8: ML-KEM encapDecap Test Case JSON Object

The following is an example JSON object sent from the server to the client for ML-KEM / encapDecap / FIPS203.

```
[
  {
    "acvVersion": <acvp-version>
  },
  {
    "vsId": 0,
    "algorithm": "ML-KEM",
    "mode": "encapDecap",
    "revision": "FIPS203",
    "testGroups": [
      {
        "tgId": 1,
        "testType": "AFT",
        "parameterSet": "ML-KEM-512",
```

```

    "function": "encapsulation",
    "tests": [
      {
        "tcId": 1,
        "ek": "75A72D...",
        "m": "013FD9A..."
      },
    ],
  },
},

```

Celi
Internet-Draft

Expires 18 October 2026
ACVP ML-KEM

[Page 15]
April 2026

```

{
  "tgId": 2,
  "testType": "VAL",
  "parameterSet": "ML-KEM-512",
  "function": "decapsulation",
  "tests": [
    {
      "tcId": 76,
      "dk": "DF479939FAA...",
      "c": "49D873A9..."
    }
  ],
},
{
  "tgId": 3,
  "testType": "VAL",
  "parameterSet": "ML-KEM-512",
  "function": "encapsulationKeyCheck",
  "tests": [
    {
      "tcId": 100,
      "ek": "ABCD..."
    }
  ],
},
{
  "tgId": 4,
  "testType": "VAL",
  "parameterSet": "ML-KEM-512",
  "function": "decapsulationKeyCheck",
  "tests": [
    {
      "tcId": 115,
      "dk": "ABCD..."
    }
  ],
},
]
}
]

```

9. Test Vector Responses

After the ACVP client downloads and processes a vector set, it must send the response vectors back to the ACVP server. The following table describes the JSON object that represents a vector set response.

Celi

Expires 18 October 2026

[Page 16]

JSON Property	Description	JSON Type
acvVersion	The ACVP version used	string
vsId	The vector set identifier	integer
testGroups	The test group objects in the response, see Table 10	array

Table 9: Response JSON Object

An example of this is the following

```
{
  "acvVersion": "version",
  "vsId": 1,
  "testGroups": [ ... ]
}
```

The 'testGroups' section is used to organize the ACVP client response in a similar manner to how it distributes vectors.

JSON Property	Description	JSON Type
tgId	The test group identifier	integer
tests	The test case objects in the response, depending on the algorithm see Table 11 or Table 13	array

Table 10: Response Group Objects

An example of this is the following

```
{
  "tgId": 1,
  "tests": [ ... ]
}
```

9.1. ML-KEM keyGen Test Vector Responses

Each test group contains an array of one or more test cases. Each test case is a JSON object that represents a single test vector to be processed by the ACVP client. The following table describes the JSON elements for each ML-KEM / keyGen / FIPS203 test vector.

JSON Value	Description	JSON type
------------	-------------	-----------

tcId	The test case identifier	integer
ek	The computed encapsulation key	hex
dk	The computed decapsulation key	hex

Table 11: ML-KEM keyGen Test Case Response JSON Object

The following is an example JSON test vector response object for ML-KEM / keyGen / FIPS203.

```
[
  {
    "acvVersion": <acvp-version>
  },
  {
    "vsId": 0,
    "testGroups": [
      {
        "tgId": 1,
        "tests": [
          {
            "tcId": 1,
            "ek": "238B9517653...",
            "dk": "0A851B79883..."
          }
        ]
      }
    ]
  }
]
```

9.2. ML-KEM encapDecap Test Vector Responses

The test groups for ML-KEM / encapDecap / FIPS203 contain public key properties. The groups can be described using the following table.

Celi Expires 18 October 2026 [Page 18]
Internet-Draft ACVP ML-KEM April 2026

JSON Value	Description	JSON type
tgId	The test group identifier	integer
tests	The individual test cases for the group	array

Table 12: ML-KEM encapDecap Test Group Response JSON Object

Each test group contains an array of one or more test cases. Each test case is a JSON object that represents a single test vector to be processed by the ACVP client. The following table describes the JSON elements for each ML-KEM / encapDecap / FIPS203 test vector.

JSON Value	Description	JSON type
tcId	The test case identifier	integer
c	The generated ciphertext only for	hex

	when function is "encapsulation"	
k	The generated shared secret for when function is "decapsulation"	hex
testPassed	Whether the key was valid when function is "encapsulationKeyCheck" or "decapsulationKeyCheck"	boolean

Table 13: ML-KEM encapDecap Test Case Response JSON Object

The following is an example JSON test vector response object for ML-KEM / encapDecap / FIPS203.

Celi

Expires 18 October 2026

[Page 19]

Internet-Draft

ACVP ML-KEM

April 2026

```
[
  {
    "acvVersion": <acvp-version>
  },
  {
    "vsId": 1564,
    "testGroups": [
      {
        "tgId": 1,
        "tests": [
          {
            "tcId": 1,
            "c": "7DFC404E2...",
            "k": "5C82A1AB3..."
          }
        ]
      },
      {
        "tgId": 2,
        "tests": [
          {
            "tcId": 76,
            "k": "87B9103AA..."
          }
        ]
      },
      {
        "tgId": 3,
        "tests": [
          {
            "tcId": 100,
            "testPassed": false
          }
        ]
      }
    ]
  },
]
```



```
{
  "tgId": 4,
  "tests": [
    {
      "tcId": 115,
      "testPassed": true
    }
  ]
}
```

Celi Expires 18 October 2026 [Page 20]
Internet-Draft ACVP ML-KEM April 2026

10. Security Considerations

There are no additional security considerations outside of those outlined in the ACVP document.

11. IANA Considerations

This document does not require any action by IANA.

12. Normative References

- [RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", RFC 2119, RFC 2119, DOI 10.17487/RFC2119, March 1997, <<https://www.rfc-editor.org/info/rfc2119>>.
- [RFC8174] Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", RFC 8174, RFC 8174, DOI 10.17487/RFC8174, May 2017, <<https://www.rfc-editor.org/info/rfc8174>>.
- [FIPS203] National Institute of Standards and Technology, "Module-Lattice-Based Key-Encapsulation Mechanism Standard", NIST FIPS 203, August 2024, <<https://csrc.nist.gov/pubs/fips/203/final>>.
- [ACVP] Fussell, B., Vassilev, A., and H. Booth, "Automatic Cryptographic Validation Protocol", ACVP, 1 July 2019.

Author's Address

Christopher Celi (editor)
Email: christopher.celi@nist.gov

Celi

Expires 18 October 2026

[Page 21]